2025

应用专题-第三方交互 07：天软平台和 Python 的交互

TSDN

深圳天软科技

本文档所有内容均属于深圳市天软科技开发有限公司的商业机密，深圳市天软科技开发有限公司亦保留上述文档中阐述的包括而不限于技术、方法、思想的所有权。此文档仅只许可因本文档所必需传播的范围内使用（仅限于内部），本文档的获得者不得将本文档散发给不相关的其他机构和个人。

深圳天软科技 |深圳市福田区新闻路 59 号深茂商业中心 19 楼

1 更新日志

1.1 更新日志

更新日期	更新说明
2026-03-23	1、 调整文档结构 2、 补充 ConnectServer 接口支持代理参数的说明 3、 新增 FAQ：Python 调用天软，将天软模块目录添加到 Python 模块搜索路径
2025-09-24	新增 FAQ：天软调用 Python 常见报错
2025-03-27	新增对 Python 3.13、Python3.14 支持
2024-11-25	1、 调整文档结构 2、 修改部分范例
2024-03-28	Python 调用 TSL 支持直接加载模块，无需修改包名
2023-11-30	修正 PyRun 说明并添加范例
2023-07-06	1、 TSL 调用 Python 中配置新增 unicode 选项 2、 TSL 调用 Python 中 PyCall 函数支持命名参数 3、 新增 TSL 调用 Python sklearn 机器学习接口
2021-04-12	1、 新增 FAQ：天软调用 Python 时不支持网格调用 2、 补充 Python 调天软时报错返回的具体报错码及对应的错误信息
2021-01-18	1、 新增常见交互异常问题的处理 FAQ 2、 补充天软调 Python 的重要配置步骤 3、 补充 Python 转码参考实现模型 tsbytestostr 对非法字符的异常处理（忽略非法字符）
2020-11-05	调整文档重点说明
2020-07-23	新增 TSL 数据转换方法：Exportjsonstring，将天软返回结果转换为 JSON 字符串
2019-12-20	修改落地服务配置步骤，新增某些场景下的配置及异常说明
2019-08-19	1、 新增 TSL 调用 Python 的函数 PyError()说明及范例 2、 新增注意事项：本地 Python 版本切换注意系统环境变量 PYTHONHOME 的更改 3、 新增落地服务器开启 Python 服务的配置方式及范例
2019-07-25	1、 新增 Python 调用天软的两个取数范例： 3.3.2 两种方式取截面数据包括基本面数据，财务数据，行情数据 3.3.3 两种方式取行情数据的时间序列数据 2、 新增天软调用 Python 时重载 Python 模块，清除变量的方法及范例
2019-07-15	新增 TSL 通过 do Python 方式执行 Python 脚本
2019-05-08	1、 添加 Python 调用天软配置说明 2、 添加 PYTHONPATH 配置说明 3、 添加字符串转换 Python2 版本限制说明
2018-01-05	添加 byte 类型转换为字符串方法到附录
2016-02-03	添加说明
2015-09-14	TSLPy2.pyd、TSLPy3.pyd 模块支持
2014-06-23	文档创建和发布

1.2 摘要

1. 多种交互方式：
   1) Python 调用 TSL
   2) TSL 调用 Python
   3) 落地服务器开启 Python 服务
   4) 其他交互方式，不在本文档中进行说明

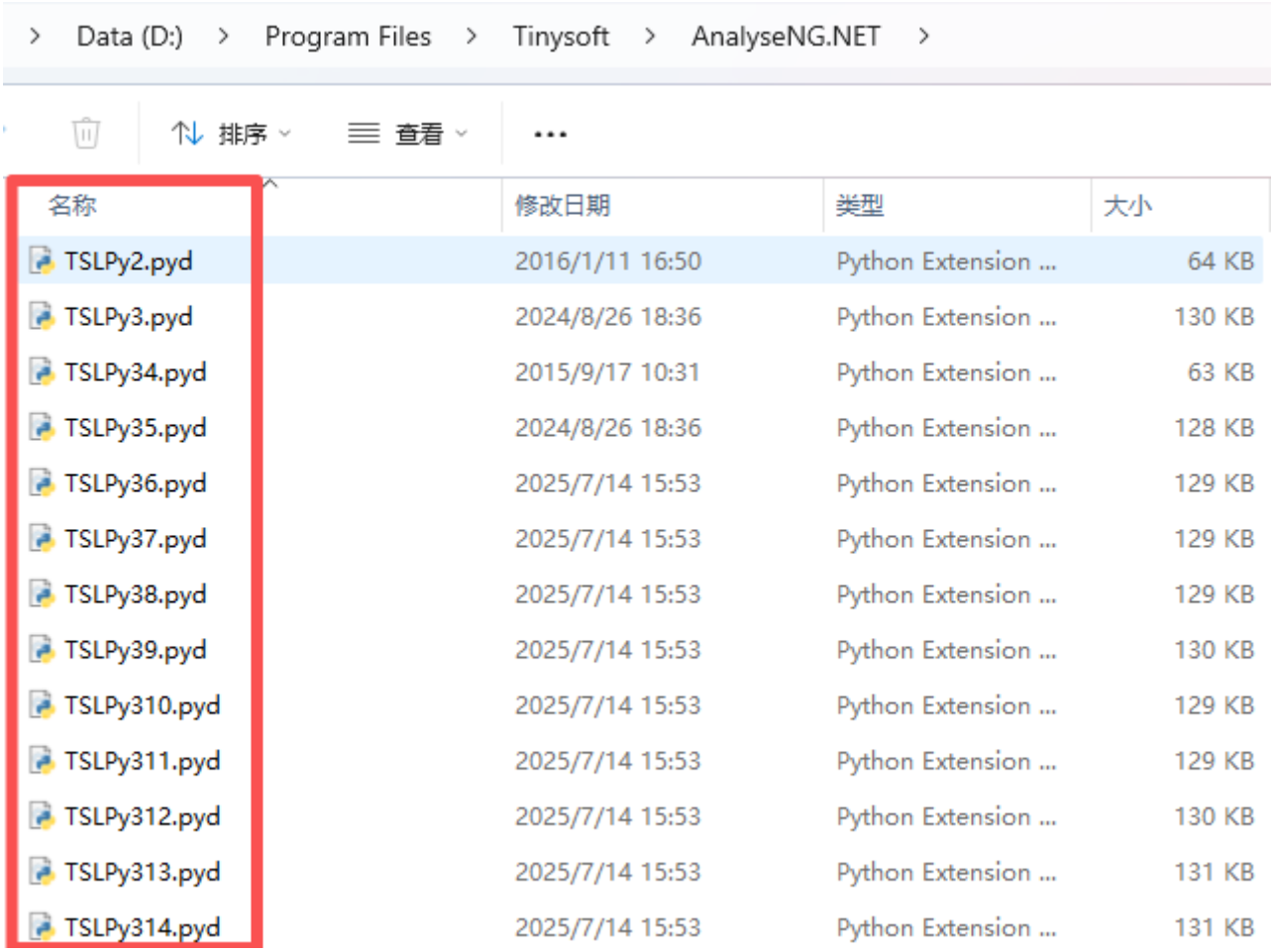
      pyTSL：天软 pyTSL 接口

      ODBC：ODBC 接口的使用方法和范例
2. 注意事项
   1) TSL 和 Python 的位数需对应，不可混用。
   2) Python 和天软的安装目录都应添加到系统环境变量中。
   3) 默认 TSLPy3.PYD 支持 Python3.4，TSLPy2.PYD 支持 Python 2.7。
   4) 基础数据类型会自动实现转换，对于非基础数据类型则需要用户自行转换。
   5) 天软调用 Python 时无法进行网格调用，详见：天软调用 Python 时不支持网格调用
   6) 某些 Python 编辑器（如 PyCharm）可能会因为缓存而导致调用失败，可以清除缓存后重试。
   7) Python 远程调用天软取数时应设置系统参数。如果远程调用无法执行，可以尝试重启 Python。
   8) 在天软客户端编辑.py 文件，将自动以 UTF8 的格式存贮，以吻合目前 Python 默认编码为 UTF8 的格式。
   9) 当 TSLPy 模块存在更新后，建议使用最新版本的 TSLPy 模块。否则可能会导致部分 TSL 调用 Python 的函数出错，例如 PyError()。
   10) 安装多个版本的 Python，如果要进行 Python 版本的切换，除了要修改 TSLPy 模块外，还需修改环境变量 PYTHONHOME，将环境变量中的 Python 路径修改为当前版本的 Python 路径。

2 支持交互的 Python 版本

截至 2026-3-23，天软支持交互的 Python 版本如下所示（**也可在天软客户端安装目录下查看**）。**特别说明：**

1) 默认 TSLPy2.pyd 支持 Python 2.7，TSLPy3.pyd 支持 Python3.4；
2) 最新支持版本为 Python3.14（即 TSLPy314.pyd），最低支持版本为 Python2.7（即 TSLPy2.pyd）与 Python3.4（即 TSLPy3.pyd、TSLPy34.pyd）；
3) 若需增加 Python 更新版本的支持，可以向天软技术支持进行反馈。



3 Python 调用 TSL

3.1 配置步骤

在 Python 调用 TSL 前需要完成以下步骤。

3.1.1 步骤一：检查位数

检查 Python 与天软的位数是否一致，需同为 64 位或者同为 32 位，不可混用。附：天软客户端安装地址：下载及试用-天软科技

具体方法：

1) Python 位数

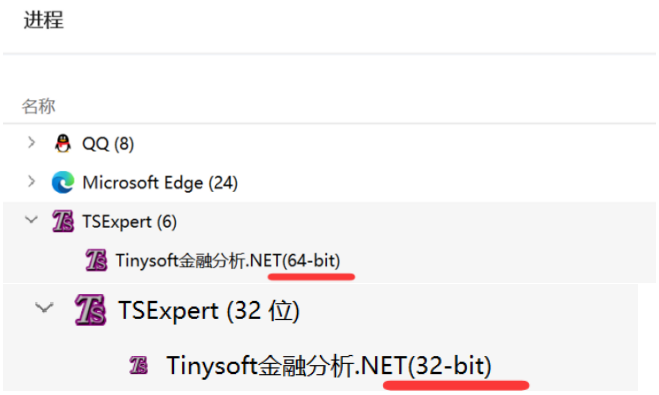
在 Python 执行以下语句：

```
import platform
print(platform.architecture())
```

如果打印元组第一个元素为'64bit'则表示 64 位，结果为'32bit'则表示 32 位。

2) 天软位数

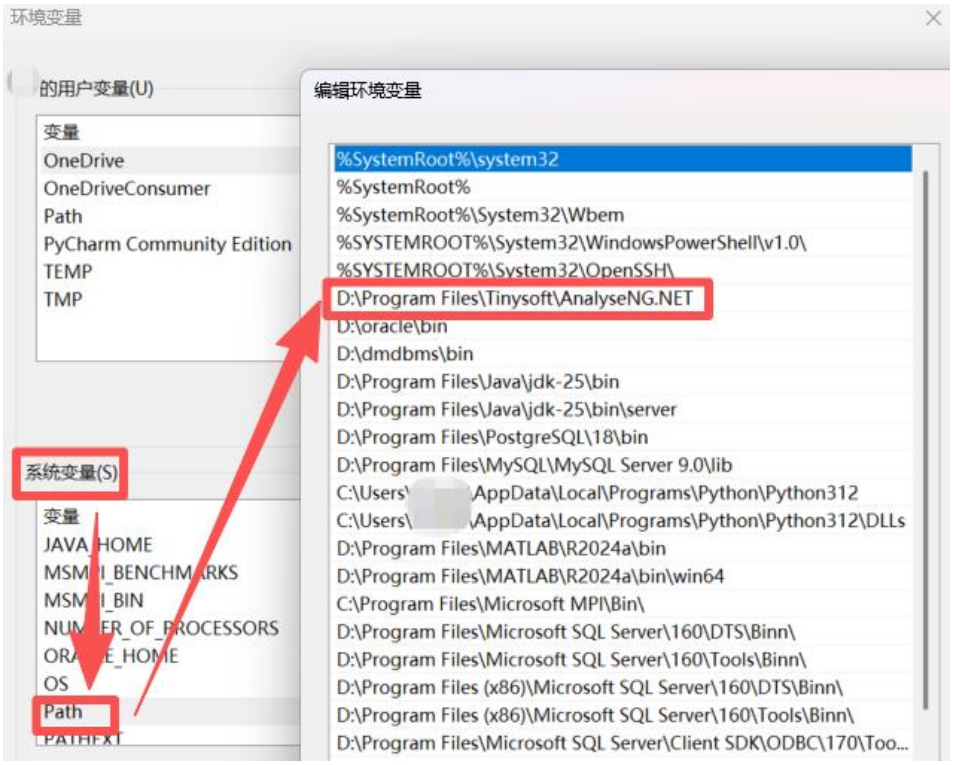
在启动天软客户端后，打开任务管理器，找到天软进程。若名称标注(64-bit)则表示 64 位，标注(32-bit)则表示 32 位。如下所示：



3.1.2 步骤二：设置系统路径

将天软客户端所在路径设置成系统路径。

具体方法：打开计算机属性-高级系统设置-环境变量-系统变量中找到 Path，点击编辑，加入天软客户端所在路径并将天软的路径加在 Path 中的第一条，如下所示：



3.1.3 步骤三：加载 TSLPy 模块

比如 Python 版本为 3.10，则直接使用 import TSLPy310 方式加载模块。执行以下语句，加载 TSLPy 模块：

```
import sys
sys.path.append("D:/Program Files/Tinysoft/AnalyseNG.NET")
import TSLPy310 as ts
```

特别注意：直接加载模块方式仅在 64 位天软客户端且版本为 2024 年之后开始支持。如果使用该方式加载报错时，可考虑**更新天软客户端版本**或者**使用修改模块包名的方式**进行加载。版本要求与加载方式支持说明如下所示：

天软客户端版本	修改模块包名方式	直接加载模块方式
新一代 64 位版本	支持	2024 年之后的版本支持，之前的不支持
64 位版本	支持	
32 位版本	支持	不支持

修改模块包名方式加载步骤：

打开天软客户端左上角系统-系统设置-打开运行/存贮目录-修改相应版本的包名。比如使用 Python3.10 请将 TSLPy310.PYD 修改为 TSLPy3.PYD，包名不区分大小写，即 TSLPy3.PYD 与 TSLPy3.pyd 并没有区别。加载语句范例如下：

```
import sys
sys.path.append("D:/Program Files/Tinysoft/AnalyseNG.NET")
import TSLPy3 as ts  #使用修改后的包名进行加载
```

注：在 Python 程序中加载对应当前 Python 版本的 TSLPy 模块，需要将模块所在目录添加到 Python 的模块搜索路径中，TSLPy 模块通常在天软客户端安装目录下。若不希望在每个 Python 程序中添加天软安装目录到模块搜索路径中，则可通过添加 PYTHONPATH 环境变量方式解决，具体可参考：Python 调用天软，将天软模块目录添加到 Python 模块搜索路径

3.1.4 步骤四：交互测试

执行以下语句，测试配置、交互是否成功。

```
import sys
sys.path.append("D:/Program Files/Tinysoft/AnalyseNG.NET")
import TSLPy3 as ts
ts.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = ts.LoginServer(user, password) #user为天软用户账号，password为用户密码
if dl[0] == 0:
    data = ts.RemoteExecute("return 1;", {})
    print(data)
else:
    print(dl[1])
ts.Disconnect() #交互完断开连接，避免占用资源
```

如果返回结果为：(0, 1, None)，则说明配置、交互成功。

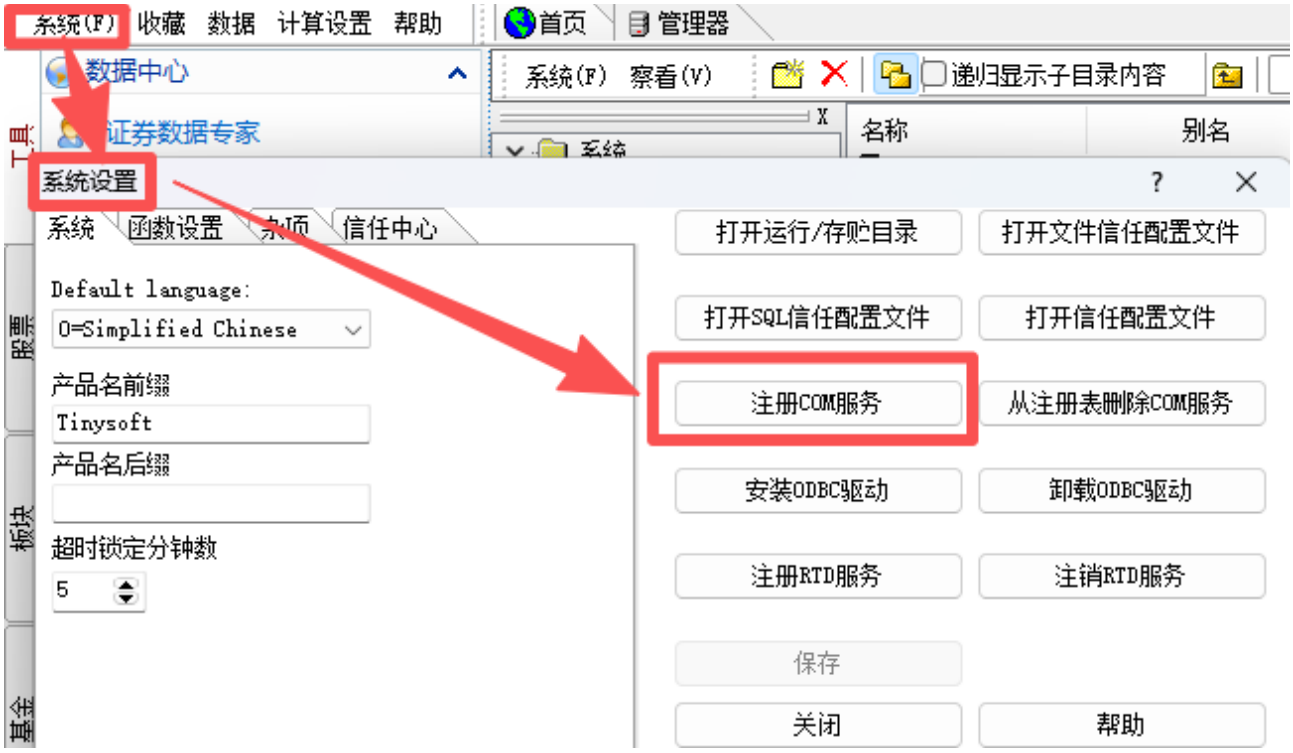
3.1.5 登录天软服务器方式介绍

天软目前提供了三种登录方式，用户按需选择其一即可，区别如下：

登录方式	优点	缺点
COM 方式	与天软客户端共用一个登录数，不额外占用登录数	需要登录天软客户端
		一次只能交互一个进程，多任务需要排队
接口方式	在 Python 程序中调用接口进行登录，无需登录天软客户端	每个连接占用一个登录数，用户账号密码信息暴露在 Python 程序中
	可同时登录多个连接	
配置文件方式	用户账号密码信息配置在本地文件中，通用于不同的 Python 程序，无需登录天软客户端	每个连接占用一个登录数
	可同时登录多个连接	

3.1.5.1 COM 方式

需要启动天软客户端（即需要打开并登录天软客户端）并注册 COM 服务。



检查注册 COM 服务是否成功可参考：检查 COM 注册是否成功

范例如下，正确结果应返回：(0, 1, None)。

```
import sys
sys.path.append("D:/Program Files/Tinysoft/AnalyseNG.NET")
import TSLPy3
data = TSLPy3.RemoteExecute("return 1;", {})
print(data)
TSLPy3.Disconnect()
```

3.1.5.2 接口方式

通过 ConnectServer 接口指定服务器地址及端口，再通过 LoginServer 接口输入用户账号、密码登录天软服务器。

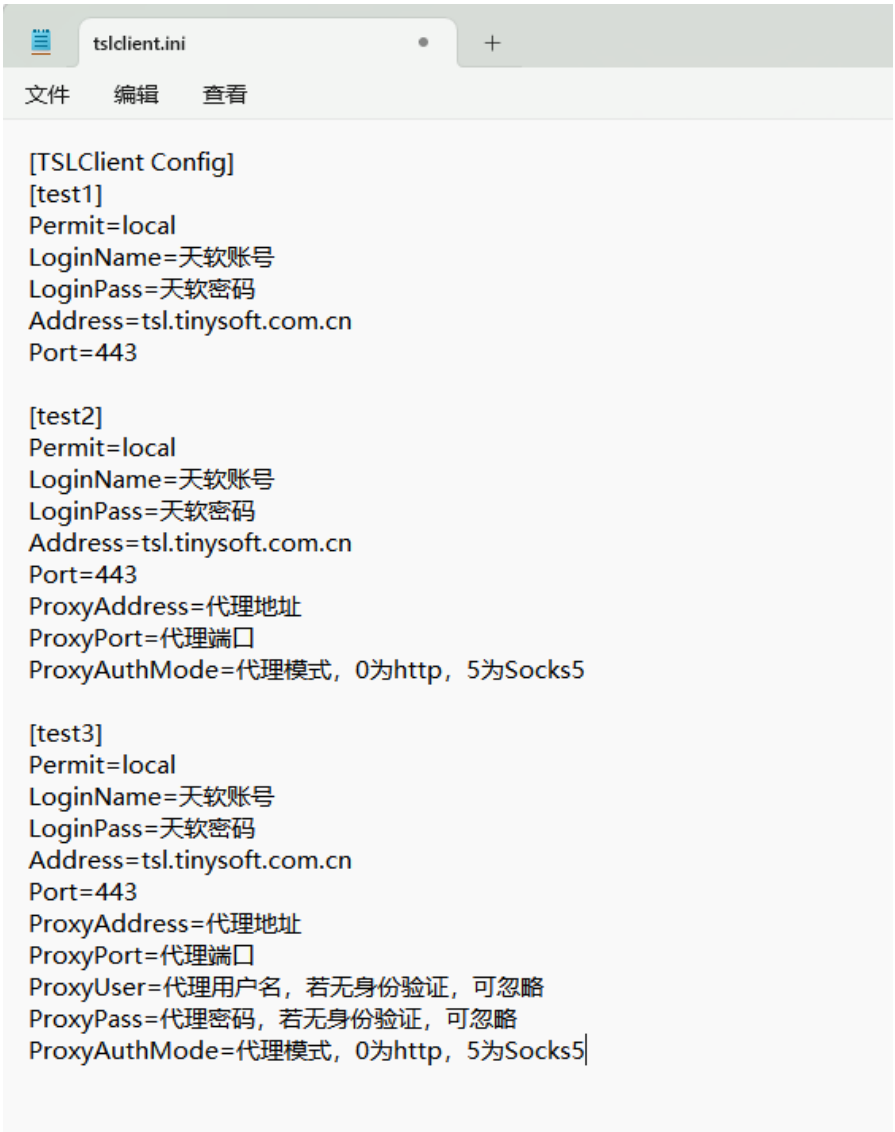
范例如下，正确结果应返回：(0, 1, None)。

```
import sys
sys.path.append("D:/Program Files/Tinysoft/AnalyseNG.NET")
import TSLPy3
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, password)  #user 为天软用户账号，password 为用户密码
if dl[0] == 0:
    data = TSLPy3.RemoteExecute("return 1;", {})
    print(data)
else:
    print(dl[1])
TSLPy3.Disconnect()
```

3.1.5.3 配置文件方式

在天软安装路径 Plugin 目录下的 tslclient.ini 配置文件中添加以下内容。**其中 Proxy 选项为代理服务器设置，若无相关需求，可省略。**

[TSLClient Config]
[别名]
Permit=local
LoginName=
LoginPass=
Address=
Port=
ProxyAddress=
ProxyPort=
ProxyUser=
ProxyPass=
ProxyAuthMode=



范例如下，正确结果应返回：(0, 1, None)。

```
import sys
sys.path.append("D:/Program Files/Tinysoft/AnalyseNG.NET")
import TSLPy3
TSLPy3.DefaultConnectAndLogin("test1")  # test1 为配置文件中的别名
data = TSLPy3.RemoteExecute("return 1;", {})
print(data)
TSLPy3.Disconnect()
```

3.2 接口说明

相关功能	接口	说明
连接、登录天软服务器	DefaultConnectAndLogin	通过配置文件连接、登录天软服务器
	ConnectServer	连接天软服务器
	LoginServer	登录天软服务器
	Disconnect	断开连接
	Logined	判断是否登录
	SetService	设置服务器类型
	GetService	获取服务器类型
	SetComputeBitsOption	设置服务器计算位数
	GetComputeBitsOption	获取服务器计算位数
执行天软语句、天软函数	RemoteExecute	远程执行天软语句
	RemoteCallFunc	远程执行天软函数
	SetSysParam	设置系统参数
	GetSysParam	获得系统参数
日期、时间数据类型的转换	EncodeDate	在 Python 中构造天软日期
	EncodeTime	在 Python 中构造天软时间
	EncodeDateTime	在 Python 中构造天软日期时间
	DecodeDate	将天软日期转换到 Python 数组
	DecodeTime	将天软时间转换到 Python 数组
	DecodeDateTime	将天软日期时间转换到 Python 数组

3.2.1 服务器相关

3.2.1.1 DefaultConnectAndLogin

- 定义：DefaultConnectAndLogin(Alias:String):Tuple
- 说明：通过配置文件登录。
- 参数：

  Alias：字符串。配置别名。

- 返回：元组。(ErrNo,ErrMsg)。当登录成功时，ErrNo 返回为 0；当登录失败时，ErrNo 返回为错误码，ErrMsg 返回为错误信息。
- 范例：

```
import TSLPy3
dl = TSLPy3.DefaultConnectAndLogin("test") # test 为配置文件中的别名
print(dl)
```

3.2.1.2 ConnectServer

- 定义：ConnectServer(Addr:String,Port:Integer,ProxyInfo:List):Integer
- 说明：连接天软服务器。如果用户是使用代理服务器进行连接，可使用 ProxyInfo 参数传入相关信息，也可参考：如何通过代理服务器登陆天软服务器。**若无相关需求，可省略。**
- 参数：

  Addr：字符串。地址。

  Port：整数。端口。

  ProxyInfo：列表。代理信息，依次为代理地址、代理端口、代理用户名、代理密码、代理模式。

  特别说明：

  1) 代理模式。目前天软已支持 http 代理（默认方式）及 SOCKS5 代理。其取值分别为 **0（http 代理）、5（SOCKS5 代理）**；
  2) 若无身份验证，可省略用户名及密码。例如：
     http 代理无身份验证时，ProxyInfo=[ip, port, 0]
     SOCKS5 代理有身份验证时，ProxyInfo=[ip, port, user, pass, 5]
- 返回：整数。当连接成功时，返回为 0；当连接失败时，返回为错误码。
- 范例：

1) 使用代理时，

```
import TSLPy3
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443, ["xxx.xxx.x.xxx", xxx, 0])
dl = TSLPy3.LoginServer(user, pass)
print(dl)
```

2) 不使用代理时，参考 LoginServer。

3.2.1.3 LoginServer

- 定义：LoginServer(User:String,Pass:String):Tuple
- 说明：登录天软服务器。占用登录数相关说明可参考：TSLPy 模块中，通过 ConnectServer 多次连接服务器算几个登陆数？
- 参数：

  User：字符串。用户账号。

  Pass：字符串。用户密码。

- 返回：元组。(ErrNo,ErrMsg)。当登录成功时，ErrNo 返回为 0；当登录失败时，ErrNo 返回为错误码，ErrMsg 返回为错误信息，如下所示：

错误码	错误信息
-1	用户无效，因安全等级要求，已经被错误的用户或者密码替代，-1 表示特殊系统性错误
-2	口令无效，因安全等级要求，已经被错误的用户或者密码替代，-2 表示特殊系统性错误
-3	用户过期
-4	用户被禁用
-5	用户重复登录
-6	用户被锁定
-8	AD 验证、系统错误
-9	过期
-10	验证码错误
-11	老版本错误的用户或者密码
-12	验证码生成错误
-13	新版本错误的用户或者密码

- 范例：

```
import TSLPy3
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
print(dl)
```

3.2.1.4 Disconnect

- 定义：Disconnect():None
- 说明：断开连接。
- 范例：

```
import TSLPy3
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
if dl[0] == 0:
   TSLPy3.Disconnect()
   print(TSLPy3.Logined())
else:
   print(dl[1])
```

3.2.1.5 Logined

- 定义：Logined():Boolean
- 说明：判断是否登录。
- 返回：布尔值。

- 范例：

  参考 Disconnect。

3.2.1.6 SetService

- 定义：SetService(Service:String):None
- 说明：设置服务器类型。
- 参数：

  Service：字符串。指定的服务器类型，可参考：Python 中指定天软服务器执行命令
- 范例：

  参考 GetService。

3.2.1.7 GetService

- 定义：GetService():String
- 说明：获取服务器类型。
- 返回：字符串。当前服务器类型。
- 范例：

```
import TSLPy3
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
if dl[0] == 0:
   TSLPy3.SetService("新功能测试版")
   print(TSLPy3.GetService().decode('gbk'))
else:
   print(dl[1])
```

3.2.1.8 SetComputeBitsOption

- 定义：SetComputeBitsOption(Option:Integer)
- 说明：设置服务器计算位数。
- 参数：

  Option：整数。指定的服务器计算位数，32 位或者 64 位。
- 范例：

  参考 GetComputeBitsOption。

3.2.1.9 GetComputeBitsOption

- 定义：GetComputeBitsOption():Integer
- 说明：获取服务器计算位数。
- 返回：整数。当前服务器计算位数。
- 范例：

```
import TSLPy3
```

```
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
if dl[0] == 0:
    TSLPy3.SetComputeBitsOption(64)
    print(TSLPy3.GetComputeBitsOption())
else:
    print(dl[1])
```

3.2.2 执行相关

3.2.2.1 RemoteExecute

- 定义：RemoteExecute(ExecString:String,SysParam:Dict):Tuple
- 说明：远程执行天软语句。
- 参数：

  ExecString：字符串。 天软语句。

  SysParam：字典。天软执行的系统参数，参考系统参数简介。**可以为空{}，但是不可省略**。
- 返回：元组。(ErrNo,Result,ErrMsg)。ErrNo 返回如下，Result 返回为结果，ErrMsg 返回为错误信息。

返回值	说明
0	执行成功
-1	连接错误以及执行函数产生的错误
-2	在提交任务过程中产生了错误（可能是在等待结果过程中断开了）
5	结果集返回错误（可能是网格等待错误，也可能是结果集过大错误）

- 范例：

```
import TSLPy3
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, password)
if dl[0] == 0:
    data = TSLPy3.RemoteExecute("return close();", {"StockID": "SZ000002", "CurrentDate": TSLPy3.En
codeDate(2019, 4, 12), "Cycle":"日线"})
    if not data[0]:
        print(data[1])
    else:
        print(data[0], data[2].decode("gbk"))
else:
    print(dl[1])
```

3.2.2.2 RemoteCallFunc

- 定义：RemoteCallFunc(Func:String,Args:List|Tuple,SysParam:Dict):Tuple
- 说明：远程执行天软函数。
- 参数：

Func：字符串。 天软函数名。

Args：列表。天软函数的参数，**可以为空[]，但是不可省略**。注意，如果天软函数需要的参数为表达式类型，直接传递会失败，需要另外实现参数的传递。比如可以重新封装函数，然后传入字符串并将字符串转换为天软的表达式类型。

SysParam：字典。天软执行的系统参数，参考系统参数简介。**可以为空{}，但是不可省略**。

- 返回：元组。(ErrNo,Result,ErrMsg)。ErrNo 返回如下，Result 返回为结果，ErrMsg 返回为错误信息。

返回值	说明
0	执行成功
-1	连接错误以及执行函数产生的错误
-2	在提交任务过程中产生了错误（可能是在等待结果过程中断开了）
5	结果集返回错误（可能是网格等待错误，也可能是结果集过大错误）

- 范例：

```
import TSLPy3
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, password)
if dl[0] == 0:
    data = TSLPy3.RemoteCallFunc("close", [], {"StockID": "SZ000002", "CurrentDate": TSLPy3.Encode
Date(2019, 4, 12), "Cycle":"日线"})
    if not data[0]:
        print(data[1])
    else:
        print(data[0], data[2].decode("gbk"))
else:
    print(dl[1])
```

3.2.2.3 SetSysParam

- 定义：SetSysParam(Name:String,Value:Any):None
- 说明：设置系统参数。
- 参数：

  Name：字符串。系统参数名。

  Value：设置系统参数的值。

- 返回：None。注意，该函数只能在本地调用，无法将设置的参数传到天软服务器，即其设置的参数无法影响 RemoteExecute 和 RemoteCallFunc 中设置的参数。
- 范例：

  参考 GetSysParam。

3.2.2.4 GetSysParam

- 定义：GetSysParam(Name:String):Any
- 说明：获得系统参数。

- 参数：

  Name：字符串。系统参数名。

- 返回：系统参数的值。注意，该函数只能在本地调用，无法用此函数获取天软服务器上的参数，即其无法获取 RemoteExecute 和 RemoteCallFunc 中设置的参数。

- 范例：

```
import TSLPy3
print(TSLPy3.SetSysParam("abc", 123))
print(TSLPy3.GetSysParam("abc"))
```

3.2.3 日期转换相关

3.2.3.1 EncodeDate

- 定义：EncodeDate(Y:Integer,M:Integer,D:Integer):Float
- 说明：在 Python 中构造天软日期。Q：天软的日期是什么格式的？
- 参数：

  Y：整数。年。
  M：整数。月。
  D：整数。日。

- 返回：浮点数。天软日期。
- 范例：

```
import TSLPy3
print(TSLPy3.EncodeDate(2019, 4, 12))
```

3.2.3.2 EncodeTime

- 定义：EncodeTime(h:Integer,m:Integer,s:Integer,ss:Integer):Float
- 说明：在 Python 中构造天软时间。
- 参数：

  h：整数。小时。
  m：整数。分钟。
  s：整数。秒。
  ss：整数。毫秒。

- 返回：浮点数。天软时间。
- 范例：

```
import TSLPy3
print(TSLPy3.EncodeTime(9, 15, 00, 100))
```

3.2.3.3 EncodeDateTime

- 定义：EncodeDateTime(Y:Integer,M:Integer,D:Integer,h:Integer,m:Integer,s:Integer,ss:Integer):Float

- 说明：在 Python 中构造天软日期时间。
- 参数：

  Y：整数。年。
  M：整数。月。
  D：整数。日。
  h：整数。小时。
  m：整数。分钟。
  s：整数。秒。
  ss：整数。毫秒。
- 返回：浮点数。天软日期时间。
- 范例：

```
import TSLPy3
print(TSLPy3.EncodeDateTime(2019, 4, 12, 9, 15, 00, 100))
```

3.2.3.4 DecodeDate

- 定义：DecodeDate(date:Float):Tuple
- 说明：将天软日期转换到 Python 数组。
- 参数：

  date：浮点数。天软日期。
- 返回：元组。(Y,M,D)，即年、月、日。
- 范例：

```
import TSLPy3
print(TSLPy3.DecodeDate(43567.0))
```

3.2.3.5 DecodeTime

- 定义：DecodeTime(time:Float):Tuple
- 说明：将天软时间转换到 Python 数组。
- 参数：

  time：浮点数。天软时间。
- 返回：元组。(h,m,s,ss)，即小时、分钟、秒、毫秒。
- 范例：

```
import TSLPy3
print(TSLPy3.DecodeTime(0.38541782407))
```

3.2.3.6 DecodeDateTime

- 定义：DecodeDateTime(datetime:Float):Tuple
- 说明：将天软日期时间转换到 Python 数组。
- 参数：

datetime：浮点数。天软日期时间。

- 返回：元组。(Y,M,D,h,m,s,ss)，即年、月、日、小时、分钟、秒、毫秒。
- 范例：

```
import TSLPy3
print(TSLPy3.DecodeDateTime(43567.38541782407))
```

3.3 范例

3.3.1 例 1：取个股指定日收盘价

取万科 A SZ000002 在 20190412 的收盘价

3.3.1.1 方式一：调用天软语句串

```
import TSLPy3
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
if dl[0] == 0:
    data = TSLPy3.RemoteExecute("return close();", {"StockID": "SZ000002", "CurrentDate": TSLPy3.En
codeDate(2019, 4, 12), "Cycle": "日线"})
    if not data[0]:
        print(data[1])
    else:
        print(data[0], data[2].decode("gbk"))
else:
    print(dl[1])
TSLPy3.Disconnect()
```

3.3.1.2 方式二：调用天软函数

```
import TSLPy3
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
if dl[0] == 0:
    data = TSLPy3.RemoteCallFunc("close", [], {"StockID": "SZ000002", "CurrentDate": TSLPy3.Encode
Date(2019, 4, 12), "Cycle": "日线"})
    if not data[0]:
        print(data[1])
    else:
        print(data[0], data[2].decode("gbk"))
else:
    print(dl[1])
TSLPy3.Disconnect()
```

3.3.2 例 2：取多个个股指定日指标

取浦发银行 SH600000、万科 A SZ000002 在指定日的截面数据，返回：代码，名称，日期，上市日，指定日总股本，指定日最新报告期，最新报告期营业收入，最新报告期归属母公司的净利润，指定日开盘价，指定日最高价，指定日最低价，指定日收盘价，指定日成交量，指定日成交金额。

3.3.2.1 方式一：调用天软语句串

```
import json
import TSLPy3
import pandas as pd
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
if dl[0] == 0:
    endt = '20190722'
    ts_str = """stockarr:=array('SH600000','SZ000002');
      endt:=""" + endt + """T;
      a:=array();
      for i:=0 to length(stockarr)-1 do
      begin
        stk:=stockarr[i];
        setsysparam(pn_stock(),stk);
        SetSysParam(pn_date(),endt);
        SetSysParam(PN_Cycle(),cy_day());
        //指定日最新报告期
        RDate:=NewReportDateOfEndT2(endt);
        a[i]['代码']:=stk;
        a[i]['名称']:=stockname(stk);
        a[i]['指定日']:=datetostr(endt);
        a[i]['上市日']:=base(12017);
        a[i]['总股本']:=StockTotalShares(endt);
        a[i]['指定日最新报告期']:=rdate;
        a[i]['营业收入']:=report(46002,RDate);
        a[i]['归属于母公司所有者净利润']:=report(46078,RDate);
        a[i]['开盘价']:=open();
        a[i]['最高价']:=high();
        a[i]['最低价']:=low();
        a[i]['收盘价']:=close();
        a[i]['成交量']:=vol();
        a[i]['成交金额']:=amount();
      end
      //将天软的数据结果转化为 json 串再传递到 Python 中
      return exportjsonstring(a);"""
```

```
    data = TSLPy3.RemoteExecute(ts_str, {})
    if not data[0]:
        print(pd.DataFrame(json.loads(data[1])))
    else:
        print(data[0], data[2].decode("gbk"))
else:
    print(dl[1])
TSLPy3.Disconnect()
```

3.3.2.2 方式二：调用天软函数

调用事先封装好的天软函数，在天软平台中封装函数：GetData_01。

- 定义：GetData_01(StockArr:Array,Endt:TDatetime):Array
- 参数：

  StockArr：数组。股票列表。

  Endt：日期。指定日。

- 代码：

```
Function GetData_01(Stockarr,Endt);
Begin
 a:=array();
 for i:=0 to length(stockarr)-1 do
 begin
   stk:=stockarr[i];
   setsysparam(pn_stock(),stk);
   SetSysParam(pn_date(),endt);
   SetSysParam(PN_Cycle(),cy_day());
   RDate:=NewReportDateOfEndT2(endt);//指定日最新报告期
   a[i]['代码']:=stk;
   a[i]['名称']:=stockname(stk);
   a[i]['指定日']:=datetostr(endt);
   a[i]['上市日']:=base(12017);
   a[i]['总股本']:=StockTotalShares(endt);
   a[i]['指定日最新报告期']:=rdate;
   a[i]['营业收入']:=report(46002,RDate);
   a[i]['归属于母公司所有者净利润']:=report(46078,RDate);
   a[i]['开盘价']:=open();
   a[i]['最高价']:=high();
   a[i]['最低价']:=low();
   a[i]['收盘价']:=close();
   a[i]['成交量']:=vol();
   a[i]['成交金额']:=amount();
 end
```

```
    return exportjsonstring(a);//将天软的数据结果转化为 json 串再传递到 Python 中
End;
```

在 Python 中调用该函数，实现代码：

```
import json
import TSLPy3
import pandas as pd
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
if dl[0] == 0:
    stockarr = ['SH600000', 'SZ000002']
    endt = TSLPy3.EncodeDate(2019, 7, 22)
    data = TSLPy3.RemoteCallFunc("GetData_01", [stockarr, endt], {})
    if not data[0]:
        print(pd.DataFrame(json.loads(data[1])))
    else:
        print(data[0], data[2].decode("gbk"))
else:
    print(dl[1])
TSLPy3.Disconnect()
```

3.3.3 例 3：取个股时间序列行情

取浦发银行 SH600000 在一段时间内的日线数据

3.3.3.1 方式一：调用天软语句串

```
import json
import TSLPy3
import pandas as pd
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
if dl[0] == 0:
    stockid = 'SH600000'
    Begt = 20190701
    Endt = 20190710
    ts_str = """ stockid:='%s';
            Begt:=%dT;
            Endt:=%dT;
            setsysparam(PN_Cycle(),cy_day());
            data:=select *,datetostr(['date']) as 'date' from markettable datekey Begt to Endt+0.99999 of stockid
end;
            //将天软的数据结果转化为 json 串再传递到 Python 中
            return exportjsonstring(data);
             """ % (stockid, Begt, Endt)
```

```
    data = TSLPy3.RemoteExecute(ts_str, {})
    if not data[0]:
        print(pd.DataFrame(json.loads(data[1])))
    else:
        print(data[0], data[2].decode("gbk"))
else:
    print(dl[1])
TSLPy3.Disconnect()
```

3.3.3.2 方式二：调用天软函数

调用事先封装好的天软函数，在天软平台中封装函数：GetData_02。

- 定义：GetData_02(StockID:String,Begt:Tdatetime,Endt:TDatetime):Array
- 参数：

  StockID：字符串。股票代码。

  Begt：日期。开始日。

  Endt：日期。截止日。

- 代码：

```
Function GetData_02(StockID,Begt,Endt);
Begin
 setsysparam(PN_Cycle(),cy_day());
 data:=select *,datetostr(['date']) as 'date' from markettable datekey Begt to Endt+0.99999 of StockID end;
 return exportjsonstring(data); //将天软的数据结果转化为 json 串再传递到 Python 中
End;
```

在 Python 中调用该函数，实现代码：

```
import json
import TSLPy3
import pandas as pd
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
if dl[0] == 0:
    StockID = 'SH600000'
    Begt = TSLPy3.EncodeDate(2019, 7, 1)
    Endt = TSLPy3.EncodeDate(2019, 7, 10)
    data = TSLPy3.RemoteCallFunc("GetData_02", [StockID, Begt, Endt], {})
    if not data[0]:
        print(pd.DataFrame(json.loads(data[1])))
    else:
        print(data[0], data[2].decode("gbk"))
else:
    print(dl[1])
TSLPy3.Disconnect()
```

3.3.4 例 4：交易策略验证

本范例旨在演示如何进行交易策略验证。在一段时间内，周期为日线，一篮子股票在 10 日线上穿 20 日线时买入，20 日线上穿 10 日线时卖出。

```
import json
import TSLPy3
import pandas as pd
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
if dl[0] == 0:
    tsstr = """
    Stocks:=
"SZ000001;SZ000002;SZ000004;SZ000005;SZ000006;SZ000007;SZ000008;SZ000009;SZ000010;SZ0
00011;SZ000012;SZ000014;SZ000016;SZ000017;SZ000018;SZ000019;SZ000020";
    BegT := 20151001T;
    EndT := 20160531T;
    cy := '日线';
    BuyCond := @Cross(MA(Close(),10),MA(Close(),20))=1;
    SellCond := @Cross(MA(Close(),20),MA(CLose(),10))=1;
    return exportjsonstring(TSFL_PB_TechnicalIndicator(Stocks,BegT,EndT,cy,BuyCond,SellCond));
    """
    data = TSLPy3.RemoteExecute(tsstr, {})
    if not data[0]:
        print(pd.DataFrame(json.loads(data[1])))
    else:
        print(data[0], data[2].decode("gbk"))
else:
    print(dl[1])
TSLPy3.Disconnect()
```

在以上的例子中，演示了如何取截面数据和时间序列数据的范例代码。其中既包括基本面数据，财务数据，也包括截面的行情数据和时间序列的行情数据。在交互过程中，我们建议减少调用次数，可以在天软平台事先封装好函数后再在 Python 中进行调用，这样可以减少天软语言错误，提高交互的效率和稳定性。当提取的数据量过大时，我们建议分批提取以提高取数效率。

除此之外，我们还准备了很多取数的范例。因篇幅有限，详情可参考：

1) Python 调天软取数范例 01：行情数据的提取
2) Python 调天软取数范例 02：基本面数据的提取
3) Python 调天软取数范例 03：财务数据的提取

4 TSL 调用 Python

4.1 配置步骤

4.1.1 步骤一：修改包名

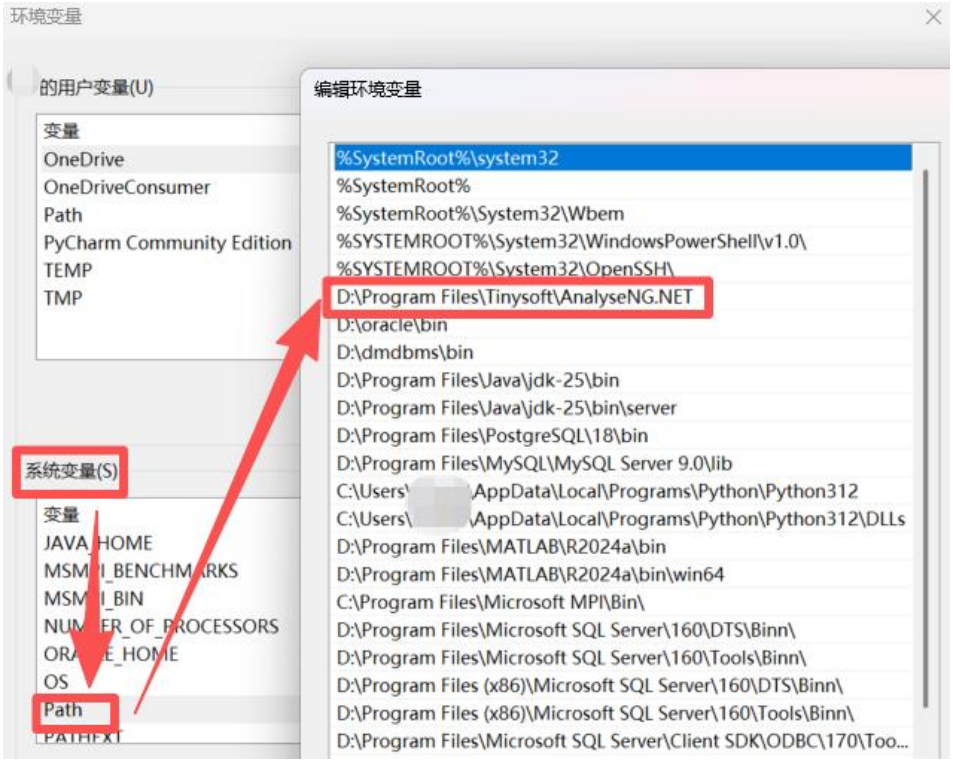
必须修改对应 Python 版本的包名。

具体方法：打开天软客户端左上角系统-系统设置-打开运行/存贮目录-修改相应版本的包名。比如使用 Python3.10 请将 TSLPy310.PYD 修改为 TSLPy3.PYD，包名 TSLPy3.PYD 与 TSLPy3.pyd 并没有区别。默认 TSLPy3.PYD 支持 Python3.4，TSLPy2.PYD 支持 Python 2.7，更多支持版本可见：支持交互的 Python 版本。

4.1.2 步骤二：设置系统环境变量

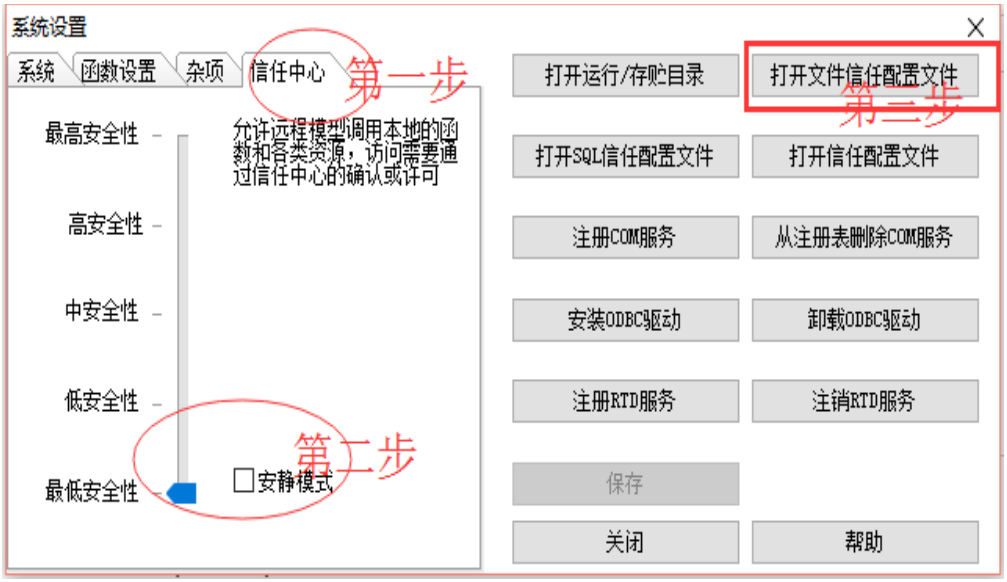
具体方法：将以下路径添加到系统环境变量 path 中，系统环境变量的添加方法可参考 http://jingyan.baidu.com/article/d5a880eb6aca7213f047cc6c.html。具体的路径如下：

1) 天软安装目录
2) Python 的 DLLs 文件路径
3) Python 的 dll 文件所在的路径



4.1.3 步骤三：配置 FileMgr.ini 文件及交互测试

1) 打开天软客户端-系统-系统设置-打开文件信任配置文件



2) 在打开的 FileMgr.ini 文件中，添加以下内容:

[Python Config]
Enabled=1
version=3
unicode=2

1) 关于 version 选项的说明：如果 Python 版本为 2.7，则 version=2；
2) 关于 unicode 选项的说明：由于天软平台的编码是 gbk，而 Python 默认为 unicode。所以在对字符的转换，特别是中文的处理时，可能会存在问题，这时可以通过设置 unicode 来解决。如果 Unicode .& 2（位与）为真，则天软的 ansi 串直接转换为 unicode 串给 Python；如果 UniCODE .& 1（位与）为真，则 Python 的 unicode 串直接转换为天软的 unicode 串。

注意，如果是在服务器端配置，添加以下内容：

[Python Config]
Enabled=1
version=3//如果 Python 版本为 2.7，则 version=2
EnginePermit=1//表明所有用户均有权限调用 Python
EnginePermitUser=User1;User2//表明 User1 和 User2 用户有权限调用 Python

3) 在客户端，新建一个函数，调用执行调用 python 代码的子函数，代码如下：

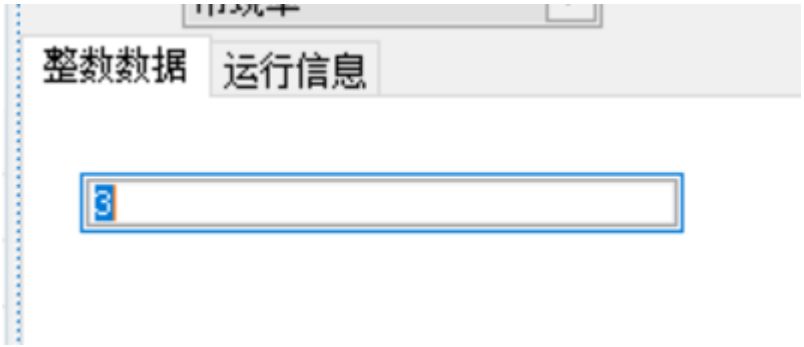
```
Function python_test();
Begin
   v:=rdo2 test();
   return v;
End;

function test();
begin
   a:=2;
   pyputvar("a",a);
```

```
    pyrun("a=a+1");
    b:=pygetvar("a");
    return b;
end
```

```
Function python_test();
Begin
    v:=rdo2 test();
    return v;
End;

function test();
begin
    a:=2;
    pyputvar("a",a);
    pyrun("a=a+1");
    b:=pygetvar("a");
    return b;
end
```



如果返回结果如上所示，则说明配置、交互成功。

4.2 接口说明

接口	说明
PyPutVar	送入 Python 变量
PyGetVar	从 Python 获得变量
PyRun	运行 Python 代码
PyCall	调用 Python 模块中的函数
PyError	获取当前运行 Python 语句时的报错信息
PyRelease	释放 Python 资源
PyEnsure	锁定 Python 资源

4.2.1 PyPutVar

- 定义：PyPutVar(VarName:String,VarValue:Any):Integer
- 说明：送入 Python 变量。
- 参数：

  VarName：字符串。变量名。

  VarValue：变量值。
- 范例：

```
a:=2;
rdo2 pyputvar("a",a);
rdo2 pyrun("a=a+1");
b:=rdo2 pygetvar("a");
return b;
```

4.2.2 PyGetVar

- 定义：PyGetVar(VarName:String):Any
- 说明：从 Python 获得变量。
- 参数：

  VarName：字符串。变量名。
- 范例：

  参考：PyPutVar。

4.2.3 PyRun

- 定义：PyRun(PythonCode:String[,flag:Integer]):Integer
- 说明：运行 Python 代码。
- 参数：

  PythonCode：字符串。Python 代码串，可执行多行代码。注意，使用空格对齐来确定逻辑层级，相同空格数代表同一层级，一般推荐使用 4 个空格。

  flag：整数。**可选参数，默认不被指定**。该参数可用于 PyCompileFlags 指定组合的状态，处理 UTF-8 源代码等问题。具体请参考 Python 手册。
- 范例：

```
s:="a=1
b=1
if a==1:
   b=2
";
rdo2 pyrun(s);
return array("a":rdo2 pygetvar("a"),"b":rdo2 pygetvar("b"));
```

4.2.4 PyCall

- 定义：PyCall([callFlag:Integer,]ModuleName:String,FuncName:String[,Param1..ParamN]):Any
- 说明：调用 Python 模块中的函数。
- 参数：

  callFlag：整数。设置结果输出方式与参数输入方式，是一个按位（2 位二进制）的状态组合值，即取值为二进制的 00，01，10，11 组合。具体含义如下：

位（从低到高位）	位值	参数值	说明
不存在		1（默认值）	直接返回调用函数的结果，参数一个个送入。
第 1 位（结果输出方式）	0	0 或 2	返回值为一个数组。 第 0 项为布尔型，表明是否调用成功。 第 1 项为调用函数的结果，若程序报错则返回错误字符串。
	1	1 或 3	直接返回调用函数的结果。 找不到 ModuleName、FuncName 则函数报错。
第 2 位（参数输入方式）	0	0 或 1	函数参数一个个送入。
	1	2 或 3	函数参数以一个数组送入。

  ModuleName：字符串。模块名。

  FuncName：字符串。模块中的函数名。Param1..ParamN 为被调用函数的参数，可以为参数数组，也可以依次传入参数。支持 Name:Value 模式进行命名参数的指定，但**依次传入的命名参数方式**目前**只在新一代客户端下一代全新测试服务器**中支持。当为参数数组时，格式为 array('参数名 1':参数值 1, '参数名 2':参数值 2, …)；当依次传入参数时，格式为参数名 1:参数值 1, 参数名 2:参数值 2,…。

- 范例：

  范例 1：数组传入参数

```
rdo2 PyRun("import numpy");
return rdo2 pycall(2,'numpy.random','randint',array('high':9,'low':6));
```

  范例 2：依次传入参数

```
rdo2 PyRun("import numpy");
return rdo2 pycall('numpy.random','randint',6,9);
```

4.2.5 PyError

- 定义：PyError():String
- 说明：获取当前运行 Python 语句时的报错信息。
- 范例：

```
rdo2 pyrun("a=math.pi");//此处会报错因为没有 import math
return rdo2 pyerror();//返回：name 'math' is not defined
```

4.2.6 PyRelease

- 定义：PyRelease():Boolean
- 说明：释放 Python 资源，多线程时可以提高效率。

4.2.7 PyEnsure

- 定义：PyEnsure():Boolean
- 说明：锁定 Python 资源。由于 PyRun、PyPutVar、PyGetVar 会智能锁定，所以用户一般不需要调用该函数。

4.3 范例

```
rdo2 PyRun("import math");
//调用 math 模块下的 sqrt(开方)函数，参数为 4
a:=rdo2 PyCall("math","sqrt",4);//a=2
//调用 math 模块下的 pow 函数，参数为 2，2
b:=rdo2 PyCall("math","pow",2,2);//b=4
return c:=a*b;//c=8
```

4.4 多线程调用 Python

4.4.1 Python 变量的全局性

天软调用的 Python 变量为全局变量，例如：

```
rdo2 pyputvar("a",'a');
test1();
test2();
return rdo2 pygetvar("a");//结果为'c'
function test1();
begin
  rdo2 pyputvar("a",'b');//修改 Python 中变量 a 的值
end

function test2();
begin
  rdo2 pyputvar("a",'c');//修改 Python 中变量 a 的值
end
```

4.4.2 注意事项

当多线程调用 Python 时，调用相关的 Python 函数会对 Python 资源进行锁定，直到该线程结束。为提高效率，当某个线程不需要调用 Python 的时候可以使用 PyRelease()函数手动对 Python 资源释放。与 PyR

elease()函数对应的还有 PyEnsure()，该函数可以锁定 Python 解释器资源。

另外，天软调用 Python 的时候暂不支持通过网格进行多线程调用，详见：天软调用 Python 时不支持网格调用 。

5 TSL 通过 do Python 方式执行 Python 脚本

在完成 TSL 调用 Python 的配置后，可以在天软客户端通过 do Python 的方式本地运行 Python 语句。但该方式不支持网格调用，详见：天软调用 Python 时不支持网格调用

5.1 语法

do extlanguage [paramlist] <scriptstring> [by config]

其中， extlanguage 指的是拓展的第三方语言。目前天软已支持 Python，MATLAB，R，今后更多的语言会被扩展支持，用户也可以在底层通过接口规范来扩展支持其他语言。本文中主要演示 do Python。

5.2 关键字

关键字	说明	示例
In	只送入变量	In A
Out	只送出变量	Out A
Var	送入送出变量	Var A
Name	重命名变量	A name "a"

注意，do Python 只能送入送出变量，执行完 Python 语句后并不能直接返回结果。因为 Python 并没有直接返回结果的 return 语句，所以需要出参来送出 Python 执行完的结果，然后在天软中使用 return 返回送出的变量。

如果使用变量名，天软会默认将同名变量名送入到外部语言中。假如目标语言大小写相关，必须用 name 关键字明确指定变量名 。

5.3 范例

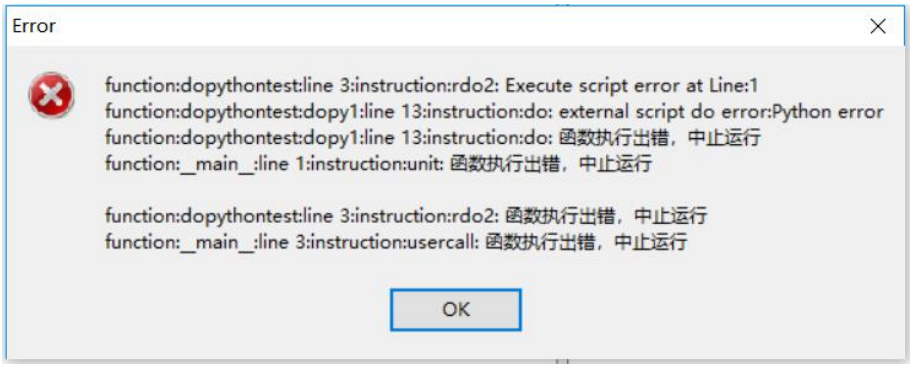
5.3.1 例 1：基本语法演示

```
Function doPythontest();
Begin
//本地执行 dopy1() 实现 do Python 执行 Python 脚本
  return rdo2 dopy1();
end

function dopy1()
begin
```

```
//do Python 的方式实现交互
 a:=10;
 b:=100;
 c:=2;
 d:=35;
 do Python in a,in b name "E",out c,var d
"c=a+E
d=d*2";
 return array(c,d);
End;
//返回[110,70]
```

注意：引号中为 Python 语法，须靠编辑器最左。有空格的话，会报 Python 错误，如下：



5.3.2 例 2：导入模块计算

此范例展示天软通过 do Python 的方式调用 Python 执行数学函数，须事先在 Python 中安装好需要的模块，例如 Scipy，math，numpy 等。未安装也会报例 1 中的错误。

```
Function KMV_DOPY(EV,St,sigma_St,F,r);
Begin
//KMV 波动率约束法
   EV:=1672130729;   //资产价值
   St:=172330000;    //股票市值
   sigma_St:=0.6179; //波动率
   F:=1499800729;    //债务
   r:=0.0425;        //无风险利率
  return rdo2 py_sub(EV,St,sigma_St,F,r);
end;

function py_sub(EV,St,sigma_St,F,r);
begin
mtic;
do Python in EV name 'EV',in St name 'St',
        in sigma_St name 'sigma_St',
        in F name 'F',
```

```
        in r name 'r', out ret
%%
from __future__ import division
from scipy.optimize import fsolve
from scipy.stats import norm
from math import pow, e, log
def KMV(EV,St,sigma_St,F,r):
    def f(x):
        #定义 fsolve 函数所需要的参数 f
        Vt = float(x[0])
        sigma = float(x[1])
        dt = float(x[2])

        def N(y):
        #定义标准正态分布的累积函数
            return norm.cdf(y)

        return [Vt/St*N(dt) - pow(e, -r)*(F/St)*N(dt - sigma) - 1,
                sigma*(Vt/St)*N(dt) - sigma_St,
                (log(Vt/F) + r + pow(sigma, 2)/2)- sigma*dt]
    x0 = [EV,0.5,1]
    result = fsolve(f,x0,epsfcn=0.5)
    result=[result[0],result[1],result[2],result[2]-result[1]]#DTD 结果为 dt-sigma
    return result
ret = KMV(EV,St,sigma_St,F,r)
%%;
echo mtoc;
  return ret;
End
//返回[1607259056.45737，0.0698146136874408，1.63483060267175，1.56501598898431]
```

6 落地服务器开启 Python 服务

6.1 工具

update(X64-yyyymmdd).rar

注意：

1) yyyymmdd 指更新包版本日期，开启此服务需要 20191101 以后版本的更新包；
2) 需安装 Python。

6.2 升级天软平台

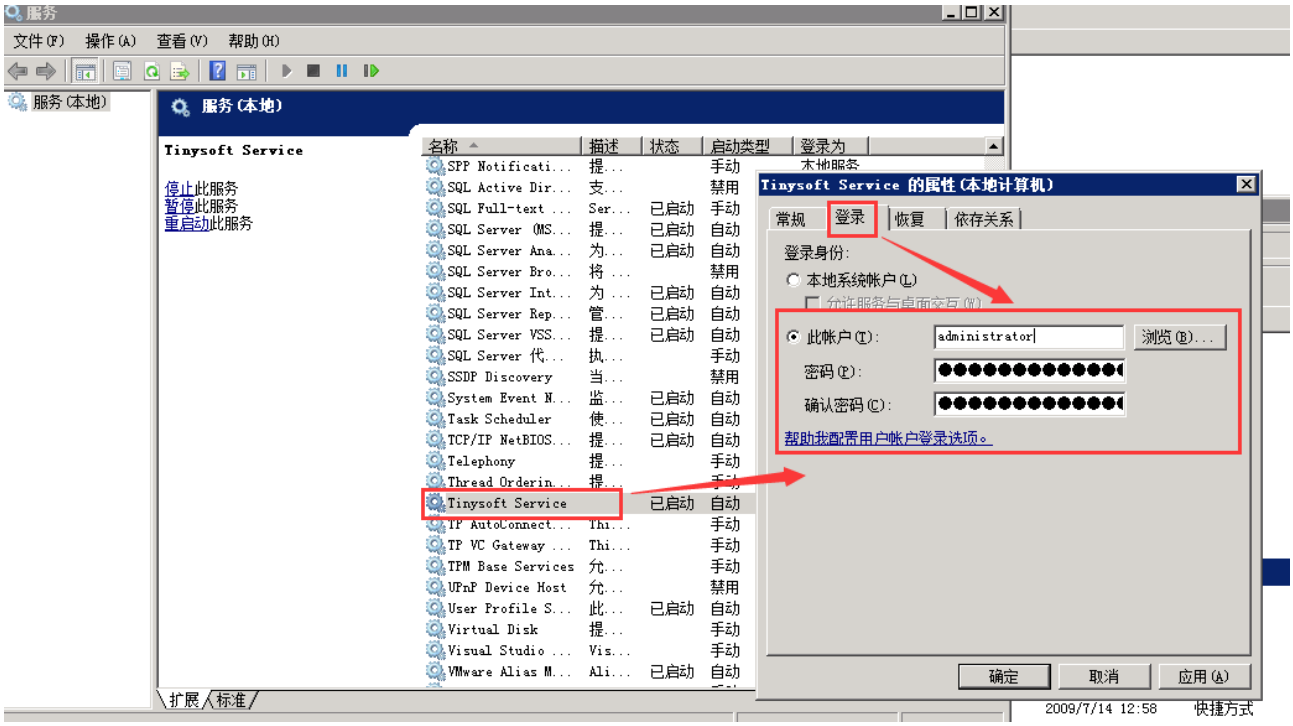
1) 将 Update(x64-yyyymmdd).rar 复制到目标盘根目录下，解压 Update(x64-yyyymmdd).rar 到目标盘根目录下生成一个名字为 update-x64 文件夹。
2) 在 tinysoft\backup\rar 中查看是否有备份文件 Tinysoft_YYYYMMDD_backup.rar，日期为最新一天。如果没有就运行\Tinysoft\backup\tinysoft_backup.cmd 进行备份。
3) 用管理员身份运行命令提示符（cmd），在命令行下，执行 net stop tssvc，停止服务。
4) 用管理员身份运行命令提示符（cmd），在命令行下，在\update-x64 目录下执行 update-x64-Level1.cmd。

6.3 服务开启配置

注意，新建的 Python 文件会在\tinysoft\bin\Pythonext\用户名\中生成，如：D:\tinysoft\bin\PythonExt\testpy\nonamepy1.py。

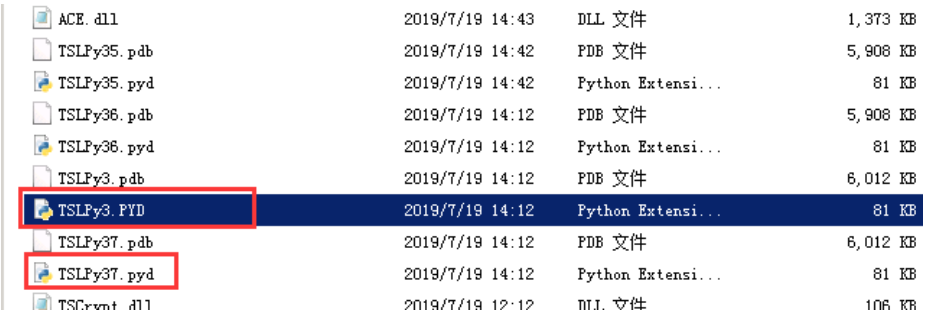
1） 升级天软平台，详情见 6.2。

2） 重启 tssvc 服务。进入控制面板-系统和安全-管理工具-服务-Tinysoft Service，右键此服务-属性-登录，选择以 administrator 或者有管理员权限的账号启动此服务。



如果不以指定账号登录，该服务将以 system 启动。而没有管理员权限的账号，在某些场景下，例如实时行情的行情源如果是以文件模式传输，则会出现异常。system 或指定账号无管理员权限的话，将无法调度文件，导致交互失败。

3） 将 Python 模块版本文件复制到\tinysoft\bin 目录下。



注意，根据 Python 版本号和子版本号选择复制对应 TSLpy.pyd 文件和.DLL 文件。例如：Python3 版本只需复制 TSLPy3.pyd，如果是 Python3.7 需要复制 TSLPy37。

4） 在\tinysoft\event\inimgr.ini 文件中，添加语句：EditorSupportPython=1。此语句是开启能将 Python 文件直接拖入天软客户端编辑器中的功能。



5） 在\tinysoft\bin\plugin\filemgr.ini 文件中，添加以下语句：

[Python Config]

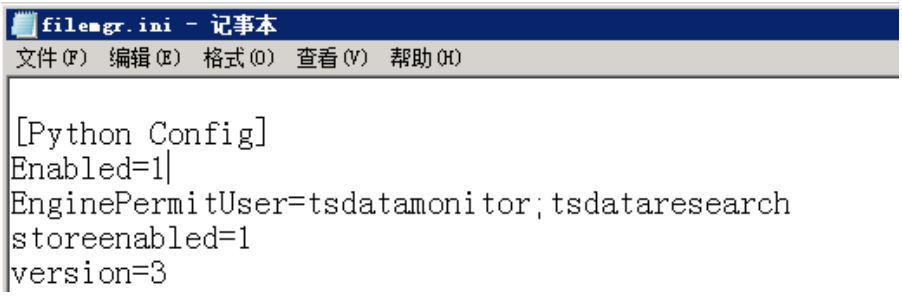
Enabled=1 注：允许打开模块

EnginePermitUser=testpy 注：授权给某一用户

storeenabled=1 注：允许保存 Python 文件到服务器

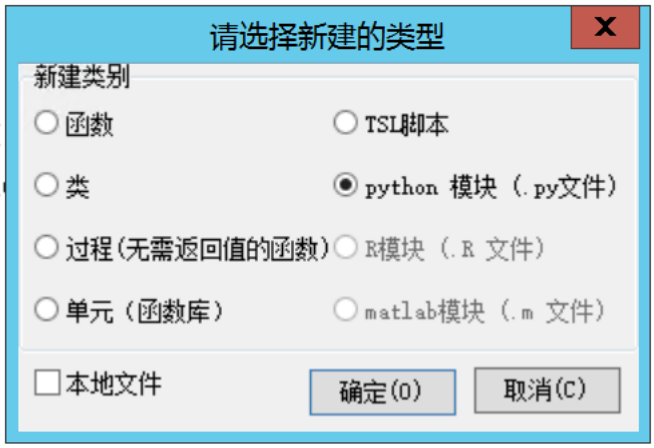
version=3 注：Python 版本号

SubVersion=7 注：Python 子版本号（非必须）



6.4 范例

6.4.1 例 1：直接在天软客户端新建 Python 模块





注意，函数名须加.py。文件所在路径：D:\tinysoft\bin\PythonExt\testpy\nonamepy1.py。也支持将 Python 文件直接拖入天软客户端中打开。在天软客户端编辑.py 文件，将自动以 UTF8 的格式存贮，以吻合目前 Python 默认编码 UTF8 格式。

6.4.2 例 2：天软平台通过 Pyrun()等函数直接调用 Python

```
Function pytest_01();
Begin
   a:=2;
   pyputvar('a',a);
   pyrun('b=a+1');
   b:=pygetvar('b');
  return b;
End;
```

6.4.3 例 3：天软平台通过 do Python 方式调用 Python

```
Function pytest_02();
```

```
Begin
 //do 的方式实现第三方交互
 a:=10;
 b:=100;
 c:=2;
 d:=35;
do Python in a,in b name "E",out c,var d
"c=a+E
d=d*3";
 return array(c,d);
end

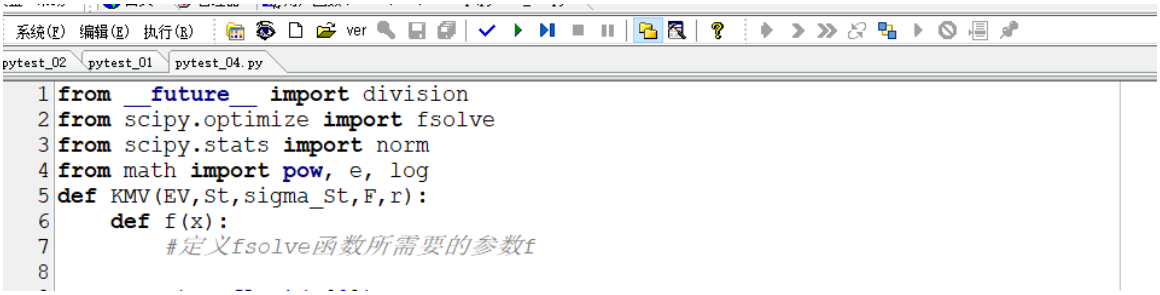
Function pytest_03();
Begin
//约束法计算 KMV 波动率
//天软取数，Python 计算
   EV:=1672130729;   //资产价值
   St:=172330000;    //股票市值
   sigma_St:=0.6179; //波动率
   F:=1499800729;    //债务
   r:=0.0425;        //无风险利率
do Python in EV name 'EV',in St name 'St',
        in sigma_St name 'sigma_St',
        in F name 'F',
        in r name 'r', out ret
%%
from __future__ import division
from scipy.optimize import fsolve
from scipy.stats import norm
from math import pow, e, log
def KMV(EV,St,sigma_St,F,r):
   def f(x):
      #定义 fsolve 函数所需要的参数 f
      Vt = float(x[0])
      sigma = float(x[1])
      dt = float(x[2])
      def N(y):
      #定义标准正态分布的累积函数
         return norm.cdf(y)

      return [Vt/St*N(dt) - pow(e, -r)*(F/St)*N(dt - sigma) - 1,
            sigma*(Vt/St)*N(dt) - sigma_St,
            (log(Vt/F) + r + pow(sigma, 2)/2)- sigma*dt]
```

```
    x0 = [EV,0.5,1]
    result = fsolve(f,x0,epsfcn=0.5)
    result=[result[0],result[1],result[2],result[2]-result[1]]#DTD 结果为 dt-sigma
    return result
ret = KMV(EV,St,sigma_St,F,r)
%%;
return ret;
end
```

6.4.4 例 4：直接调用在天软平台编辑或拖拽进入的.py 文件

先编辑.py 文件或者是直接将已经编辑好的.py 文件拖入客户端，这里举例在客户端编写 pytest_04.py 文件。




```
from __future__ import division
from scipy.optimize import fsolve
from scipy.stats import norm
from math import pow, e, log
def KMV(EV,St,sigma_St,F,r):
    def f(x):
        #定义 fsolve 函数所需要的参数 f
        Vt = float(x[0])
        sigma = float(x[1])
        dt = float(x[2])

        def N(y):
        #定义标准正态分布的累积函数
            return norm.cdf(y)

        return [Vt/St*N(dt) - pow(e, -r)*(F/St)*N(dt - sigma) - 1,
                sigma*(Vt/St)*N(dt) - sigma_St,
                (log(Vt/F) + r + pow(sigma, 2)/2)- sigma*dt]
    x0 = [EV,0.5,1]
    result = fsolve(f,x0,epsfcn=0.5)
    result=[result[0],result[1],result[2],result[2]-result[1]]#DTD 结果为 dt-sigma
    return result
ret = KMV(EV,St,sigma_St,F,r)
```

然后调用平台中编辑好的.py 文件。

```
Function pytest_05();
Begin
 //约束法计算 KMV 波动率
 //天软取数，Python 计算。PyCall 方式
   EV:=1672130729;  //资产价值
   St:=172330000;   //股票市值
   sigma_St:=0.6179; //波动率
   F:=1499800729;   //债务
   r:=0.0425;       //无风险利率
return pycall('pytest_04','KMV',EV,St,sigma_St,F,r);
End

Function pytest_06();
Begin
 //约束法计算 KMV 波动率
 //天软取数，Python 计算。do Python 方式
   EV:=1672130729;  //资产价值
   St:=172330000;   //股票市值
   sigma_St:=0.6179; //波动率
   F:=1499800729;   //债务
   r:=0.0425;       //无风险利率
do Python in EV name 'EV',in St name 'St',
      in sigma_St name 'sigma_St',
      in F name 'F',
      in r name 'r', out ret
%%
import pytest_04
ret=pytest_04.KMV(EV,St,sigma_St,F,r)
%%;
return ret;
End
```

6.4.5 例 5：在天软平台编辑.py 文件调用天软平台

编辑.py 文件 pytest_07.py，取 SH600000 今日日线行情数据。如下所示：

pytest_02 | pytest_01 | NoName19 | Function16 | pytest_07.py | pytest_03.py

```
1 # Python Script Code
2 def pyclose():
3     import TSLPy3 as ts
4     ts_str="""setsysparam(pn_CYCLE(),CY_DAY());
5                           RETURN SELECT * FROM markettable datekey
6                           TODAY() TO TODAY()+16/24 OF 'SH600000' END;"""
7     data=ts.LocalExecute(ts_str)
8     return data
```

```
# Python Script Code
def pyclose():
    import TSLPy3 as ts
    ts_str="""setsysparam(pn_CYCLE(),CY_DAY());
                RETURN SELECT * FROM markettable datekey
                TODAY() TO TODAY()+16/24 OF 'SH600000' END;"""
    data=ts.LocalExecute(ts_str)
return data
```

注意，此处需要使用 LocalExecute()，如果使用 RemoteExecute()会出错，无返回结果。同样，如果调用天软函数，需要使用 LocalCallFunc()。

在天软平台调用 pytest_07.py 文件。

```
Function NoName19();
Begin
pyrun('import sys');
//须新增服务器上保存账户下新建的.py 文件的路径，否则会报找不到模块的错误
//账户指以客户账户名命名的文件夹
pyrun("sys.path.append('D:/tinysoft/bin/PythonExt/账户')");
b:=pycall('pytest_07','pyclose');
return b;
end
```



7 TSL 调用 sklearn 机器学习接口

为了方便用户使用 sklearn 的算法，天软专门封装了对应的接口，可以像使用 TSL 代码一样直接使用 sklearn 包里面的算法。机器学习相关接口主要有 tspy_utils 和 sklearn 接口。

7.1 配置

1、 配置 TSL 与 Python 交互：TSL 调用 Python

2、 Python 中安装 sklearn 包。

7.2 使用说明及范例

tspy_utils：该单元封装了一些天软调用 Python 的方便接口，对天软与 Python 交互的底层接口进行了一定的封装。

sklearn：为方便用户使用 sklearn 的算法，天软专门封装了对应的接口，可以像使用 TSL 代码一样直接使用 sklearn 包里面的算法。单元命名 sklearn_interface_xx_xx，xx_xx 对应的是 Python 中 sklearn 本身的版本号规则中“一级版本号.二级版本号”

关于这两单元的详细说明可参看 TSL 语言帮助文档。

机器学习主要使用 sklearn 接口，封装是对类的接口进行封装，接口类的成员函数接口跟 Python 原本的接口名一致，参数只有一个，表示调用该成员函数的参数组成的数组，字符串下标表示命名参数。例如线性回归的范例代码如下：

```
uses sklearn_interface_1_2; //注意，要使用对应版本号的接口
obj := new LinearRegression();
obj.fit_intercept := false;
obj.fit(array('X':x,'y':y));
obj.coef_.tolist();
writeln(tostn(obj.predict(array(x)).tolist()|y));
```

其他更多范例可参考：sklearn_interface_demo

8 附录

8.1 数据类型转换

基础数据类型自动转换规则如下所示：

TSL	Python
NIL	None
NAN	nan
INF	inf
array	List
array	tuple
array	dict
Complex	complex

由于其他数据类型的复杂性，非基础数据类型请用户自行实现转换。例如要将 Python 中的时间类型送入到天软中，天软的 TSLPy2\TSLPy3 模块中提供了相应的转换函数。

8.2 系统参数简介

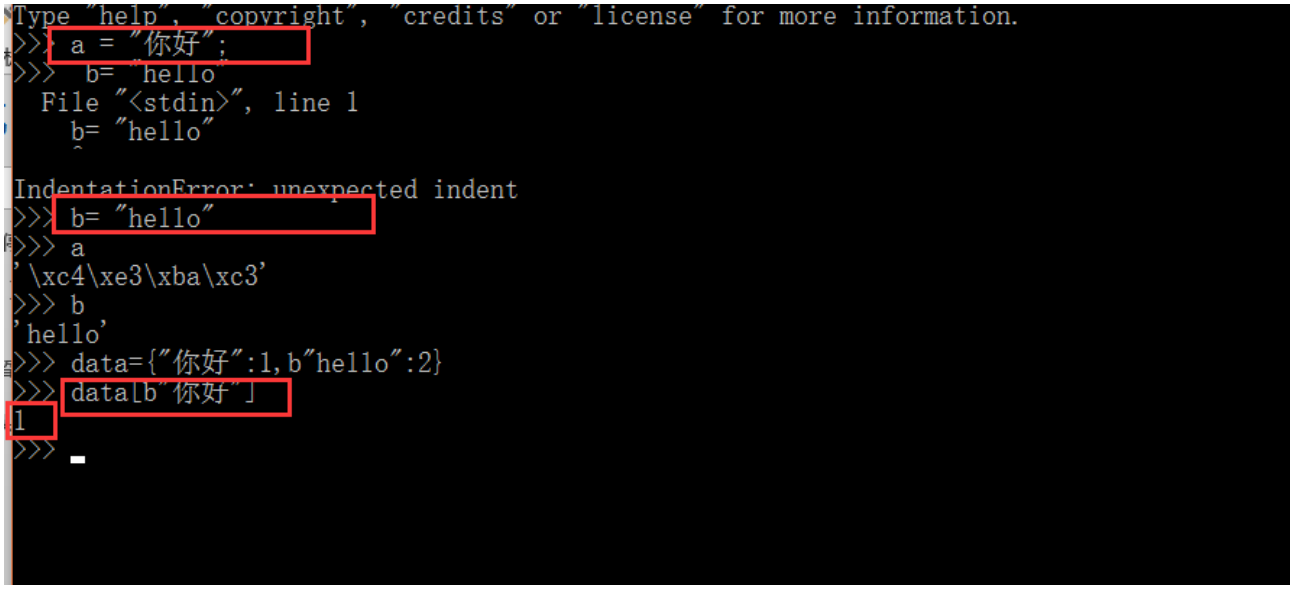
系统参数名称	含义	例子

StockID	当前股票代码	"SH000001"
CurrentDate	当前日期	TSLPy3.EncodeDate(2015,9,25)
Cycle	周期	"日线"
bRate	复权方式	0
RateDay	复权基准日	TSLPy3.EncodeDate(2015,9,25)
Precision	显示精度	4

系统参数的详细介绍可参见：系统参数的含义

8.3 数据结构转换方法（克服部分中文在 Python 中的显示问题）

首先，Python 返回的不是乱码，可参考：http://www.cnblogs.com/jxzheng/p/5186490.html。举例如下：



8.3.1 Exportjsonstring（推荐方式）

使用 Exportjsonstring 转换数据。在天软平台中将天软返回的结果直接转化成 UTF-8 编码的 JSON 字符串送入 Python，经过此方法处理后：

1、 数据量不变，占用内存小，传输速度更快；

2、 天软平台向 Python 一次最多送入 190MB 数据。如不处理，则可传输不大于 152000 行日线数据。

处理后则可增大传输的数据量，在同是 190MB 大小限制的条件下，传输更多的日线，约 660000 行日线数据，相当于数据被压缩。

举例如下，天软代码：

```
Function gettinydata();
Begin
  SetSysParam(PN_Cycle(),cy_day);
  stockarr:=GetBKByDate('SH000300',20200721T);
  data:=select * from markettable datekey 20200721T to 20200721T+0.9 of stockarr END;
```

```
    return exportjsonstring(data);
End
```

Python 代码：

```
import json
import TSLPy3
import pandas as pd
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
if dl[0] == 0:
    data = TSLPy3.RemoteCallFunc("""gettinydata""", [], {})
    if not data[0]:
        print('数据传输成功')
        print(pd.DataFrame(json.loads(data[1])))
    else:
        print('数据传输失败')
        print(data[0], data[2].decode("gbk"))
else:
    print(dl[1])
```

8.3.2 tsbytestostr（不推荐方式）

在 Python 端封装使用，不会改变天软返回结果的结构。**注意，该函数只在 Python3 版本中有效，Python2 中无效。**举例如下：

Python 代码：

```
def tsbytestostr(data):
    if isinstance(data, tuple) or isinstance(data, list):
        lendata = len(data)
        ret = []
        for i in range(lendata):
            ret.append(tsbytestostr(data[i]))
    elif isinstance(data, dict):
        ret = {}
        for i in data:
            ret[tsbytestostr(i)] = tsbytestostr(data[i])
    elif isinstance(data, bytes):
        ret = data.decode('gbk', errors='ignore')
    else:
        ret = data
    return ret

import TSLPy3
TSLPy3.ConnectServer("tsl.tinysoft.com.cn", 443)
dl = TSLPy3.LoginServer(user, pass)
```

```
if dl[0] == 0:
    data = TSLPy3.RemoteExecute("return array('股票',defaultstockid(),'名称',stockname(defaultstockid()),
'日期',datetostr(sp_time()),'收盘价',close());", {"StockID": "SZ000002", "CurrentDate": TSLPy3.Encode
Date(2019, 4, 12), "Cycle":"日线"})
    if not data[0]:
        print(tsbytestostr(data[1]))
    else:
        print(data[0], data[2].decode("gbk"))
else:
    print(dl[1])
```

8.4 本地调用天软（通常不需要使用）

8.4.1 LocalExecute

- 定义：LocalExecute(ExecString:String):Tuple
- 说明：本地执行天软语句。
- 参数：

  ExecString：字符串。天软语句。
- 返回：元组。(ErrNo,Result,ErrMsg,ErrLineNo) 。当语句执行成功时，ErrNo 返回为 0；当语句执行失败时，ErrNo 返回为错误码；Result 返回为结果；ErrMsg 返回为错误信息；ErrLineNo 返回为报错行号。

8.4.2 LocalCallFunc

- 定义：LocalCallFunc(Func:String,Args:List|Tuple):Tuple
- 说明：本地执行天软函数。
- 参数：

  Func：字符串。天软函数名称。

  Args：列表。天软函数的参数。
- 返回：元组。(ErrNo,Result,ErrMsg,ErrLineNo) 。当语句执行成功时，ErrNo 返回为 0；当语句执行失败时，ErrNo 返回为错误码；Result 返回为结果；ErrMsg 返回为错误信息；ErrLineNo 返回为报错行号。

8.5 天软调 Python：重置加载包，清除变量

在天软平台调用 Python 的时候，如果不关闭或重启客户端，则 Python 的环境不会重启。此时由于 Python 的缓存机制，上次函数执行的环境并未清除，导致上次天软平台调用 Python 时加载的包还在，传入 Python 的变量也未被清除。如有需要，可以在天软平台调用 Python 的时候清除变量，重置加载包。

8.5.1 实现代码

```
Unit tspy_unit;
Interface
  function QY_py_reloadMK(modelname);   //重载模块
  function QY_py_cleardiyVar();         //清除自定义变量
  function QY_py_trandate(endt,varname); //转换天软日期为 Python 日期
  function QY_py_getsyspath();          //获得 sys.path
  function QY_py_reset();               //重置 Python 环境
  function QY_py_sns_load_dataset(dsetname);
Implementation
//1._____________________________________________
//____重载模块_________
  function QY_py_reloadMK(modelname);
  begin
    pyrun("import imp");
    pyrun(format("imp.reload(%s)",modelname));
  end
//2._____________________________________________
//____删除模块_________
  function QY_py_delMK(modelname);
  begin
   pyrun(format("del %s",modelname));
  end
//3._____________________________________________
//____清除自定义变量_________
  function QY_py_cleardiyVar();
  begin
    pyrun('[globals().pop(key) if not key.startswith("__") else 1 for key in list(globals().keys())]');
  end
//4._____________________________________________
//____将天软日期转为 Python 日期_____
  function QY_py_trandate(endt,varname);
  begin
    //endt 日期; varname Python 中变量名
    pyrun("import datetime");
    pyrun(format("%s = datetime.date(%d,%d,%d)",
                 varname,YearOf(endt)
                 ,MonthOf(endt),dayof(endt)
                 ));
  end;
//5._____________________________________________
//____获得 sys.path______
```

```
  function QY_py_getsyspath();
  begin
    pyrun("syspath = sys.path");
    return pygetvar("syspath");
  end
//6._____________________________________________
//____重载所有模块_______
  function QY_py_reloadallMK();
  begin
    pyrun("import imp");
    pyrun("import types");
    pyrun('[imp.reload(globals()[key]) if isinstance(globals()[key],types.ModuleType) else 0 for key in
list(globals().keys())]');
  end
//7._____________________________________________
//____重置 Python 环境_____________________
  function QY_py_reset();
  begin
    QY_py_reloadallMK();//重载所有模块
    QY_py_cleardiyVar();//清理所有自变量
  end
//8._____________________________________________
//____导入 seaborn 数据集_____________________
  function QY_py_sns_load_dataset(dsetname);
  begin
    pyrun("__temp__ = None");
    pyrun("import seaborn as sns");
    pyrun(format("__temp__ = sns.load_dataset('%s').to_dict('record')",dsetname));
    return pygetvar("__temp__");
  end;
Initialization
Finalization
End.
```

8.5.2 范例

8.5.2.1 例 1：PyRun 调用

```
//导入系统模块，获取系统路径
rdo2 pyrun("import sys");
rdo2 pyrun("a=sys.path");
s1:=rdo2 pygetvar("a");
//执行清除变量，重载模块前能获取到系统路径
Rdo2 unit(tspy_unit).QY_py_reset();
```

```
//执行之后则不会获取到系统路径
rdo2 pyrun("b=sys.path");
s2:=rdo2 pygetvar("b");
s3:=rdo2 pygetvar("a");
return array(s1,s2,s3);
```

8.5.2.2 例 2：do Python 调用

```
Function dopy();
Begin
//导入系统模块，获取系统路径
  rdo2 py_test();
  s1:=rdo2 py_test1();
  Rdo2 unit(tspy_unit).QY_py_reset();
// s2:=rdo2 py_test1();
//此处如果再次执行 rdo2 py_test1();则会 Python 报错
//原因是 sys 模块被清除，Python 会报 sys 未定义的错误
  return s1;
end;

function py_test();
begin
//导入系统模块
do Python
%%
import sys%%;
end

function py_test1();
begin
//获取系统路径
do Python out a
%%
a=sys.path%%;
return a;
end
```

9 FAQ

常见错误	解决方案
Python 调用返回为 none	1.检查登陆是否成功
	2.检查客户端信任配置
	3.检查调用接口是否缺少参数

	4.Python 脚本文件请采用 gbk 编码
Python 调用登陆失败	1.检查客户端信任配置
	2.天软客户端是否添加到系统环境变量（最好放最前面）
Python 调用找不到模块	1. 直接 import 加载模块
	2. 将天软安装目录添加到 Python 的 sys.path 后重启 Python
Python 调用不是有效的 win32	确认 Python 与天软位数相同
Python 调用返回乱码	天软返回是 gbk 编码，可以用 ExportjsonString 将天软的数据结果转换为 json 串再传递到 Python 中

9.1 Q1：Python 调用天软，丢失某些 dll

可参考：python 调用天软，丢失某些 dll

9.2 Q2：Python 调用天软，返回 none 或返回错误结果

可参考：python 调用天软，返回 none 或返回错误结果

9.3 Q3：Python 调用天软，提示找不到指定模块

可参考：python 调用天软，提示找不到指定模块

9.4 Q4：Python 调用天软，提示无法连接到服务器

可参考：python 调用天软，提示无法连接到服务器

9.5 Q5：Python 调用天软，返回乱码

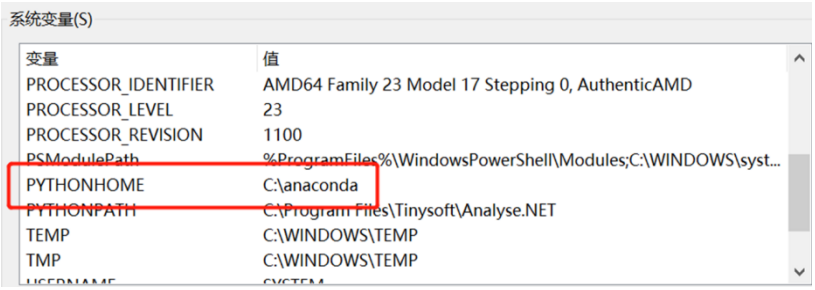
可参考：python 调用天软，返回乱码

9.6 Q6：配置了天软调 Python 后，无法启动天软客户端

解决方法：

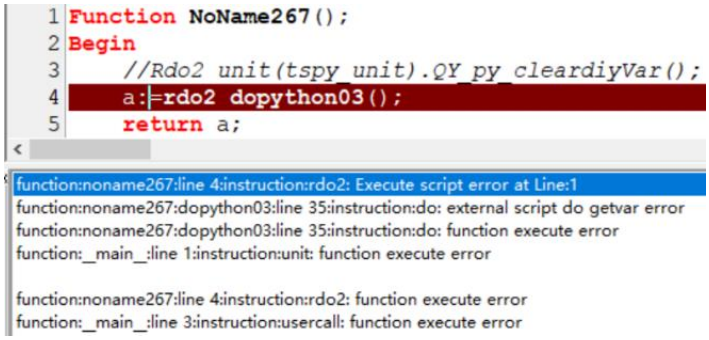
1. 以管理员权限启动天软客户端。若还不行，尝试方法 2。

2. 将 Python.exe 所在路径添加到系统环境变量 PYTHONHOME 中。客户一般安装的是集成环境下的 Python，比如 Anaconda，容易引发此异常。比如 Python.exe 在路径 c:\anaconda 下，添加至系统环境变量 PYTHONHOME 如下所示：



9.7 Q7：天软调用 Python 报错后，重新执行正确代码仍然报错

比如报错如下：



原因：由于 Python 的缓存机制，上次函数执行传入 Python 的变量未被清除，导致天软平台再次调用 Python 时变量异常的问题还存在。

解决方法：用户可将 8.5 重置加载包，清除变量中提供的清除缓存的实现单元 tspy_unit 在天软中封装并调用，清除相关变量缓存后再执行正确代码。实现如下：

1. 天软中封装 tspy_unit



2. 调用 tspy_unit 清除变量缓存，重新执行正确代码

[code]

```
Rdo2 unit(tspy_unit).QY_py_cleardiyVar();
a:=rdo2 doPython03();//doPython03()中错误代码已修正
return a;
```

[/code]

9.8 Q8：天软调用 Python 时不支持网格调用

因为 Python 本身的多线程是不安全的，在进行多线程并发的时候，由于内存公用，可能会出现在执行 N 条之后多个线程在缓存池内修改同一变量的情况，导致出现脏数据，使得线程计算出错误的结果。当然，我们可以在 Python 中使用 threading 模块中的互斥锁（lock）/递归锁（Rlock）/信号量（BoundedSemaphore）等类来锁住变量，守护线程安全。

而天软调用 Python 时，无论是使用 PyRun/do Python，或是落地服务器调用 Python 脚本，并不能在网格调用的时候有效的锁住每个线程所需的变量。这是由于 Python 本身的缓存机制问题，网格只是简单的将 Python 代码并发地送入 Python 执行，同样有可能会出现多个线程修改同一变量的情况，导致在网格调用时计算出错误的结果。

即使在天软调用 Python 的 Python 代码中加入 threading 模块的功能以作限制，如互斥锁之类的功能确实能在线程中锁住变量。但网格是并发的，在上一个线程中锁住了变量，将导致同时刻的其他线程无法调用该变量，从而产生网格报错，中断程序运行。

综上，天软调用 Python 时不支持网格调用。

9.9 Q9：天软调用 Python 常见报错

比如报错如下：

1. 天软通过 PyPutVar 等接口调用 Python
   1) 普通客户端报错

```
D:\Program Files\Tinysoft\Analyse.NET>tsl.exe test.tsl
2025-09-24 11:02:34 Load Trade GateWay(TT_CTP_GATEWAY.DLL) ok.
Execute script error at Line:2
function:__main__:line 2:instruction:usercall: 执行函数 pyputvar 出错。
function:__main__:line 2:instruction:usercall: function execute error
```

   2) NG 客户端报错

```
C:\Program Files\Tinysoft\AnalyseNG.NET>tsl.exe test.tsl
2025-09-24 11:06:21 Load Trade GateWay(TT_CTP_GATEWAY.DLL) ok.
loading TSNetDB TSL plugin...
Execute script error at Line:2
function:__main__:line 2:instruction:usercall: function:pyputvar compile error or not found
function:__main__:line 2:instruction:usercall: Function execution error
```

2. 天软通过 do Python 方式执行 Python 脚本
   1) 普通客户端报错

```
D:\Program Files\Tinysoft\Analyse.NET>tsl.exe tsl.tsl
2025-09-24 11:16:10 Load Trade GateWay(TT_CTP_GATEWAY.DLL) ok.
Execute script error at Line:5
function:__main__:line 5:instruction:do: no rights to do external script
function:__main__:line 5:instruction:do: function execute error
```

   2) NG 客户端报错

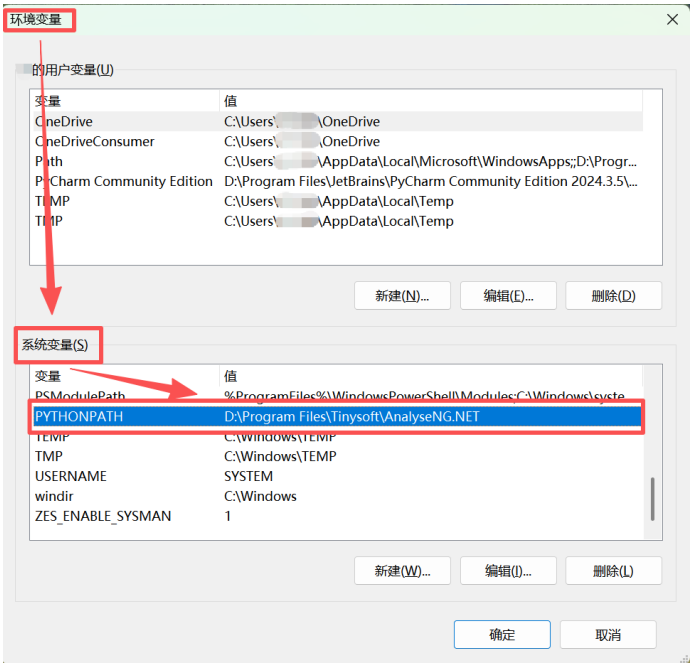
```
C:\Program Files\Tinysoft\AnalyseNG.NET>tsl.exe tsl.tsl
2025-09-24 11:12:11 Load Trade GateWay(TT_CTP_GATEWAY.DLL) ok.
loading TSNetDB TSL plugin...
Execute script error at Line:5
function:__main__:line 5:instruction:do: External script type [Python]has not been registered!
function:__main__:line 5:instruction:do: Function execution error
```

原因：可能是配置时没有正确修改 TSLPy.PYD 文件的包名。

解决方法：用户可检查配置步骤，确保配置已成功，再进一步检查交互代码是否存在语法问题。

9.10 Q10：Python 调用天软，将天软模块目录添加到 Python 模块搜索路径

在系统环境变量中新建名为 PYTHONPATH 的变量，值为天软所在安装路径。如下所示：



这种方式不用在调用天软的 Python 程序中加上 import sys sys.path.append("D:/Program Files/Tinysoft/AnalyseNG.NET")语句，直接 import TSLPy3 即可。